Remote Process Manager

Sistema de Administración de Procesos Distribuido

Reporte Técnico de Proyecto

Fernando Israel Rios Garzia
Angel Cuevas Lopez
Artur Mikhail Lazurenko Mannanov

Sistemas Operativos

Ana Cruz

28 de febrero de 2026

Proyecto desarrollado para la asignatura de Sistemas Operativos

Índice general

1. Introducción **3**
1.1. Descripción General . . . 3
1.2. Motivación y Contexto . . . 3
1.3. Alcance del Proyecto . . . 3

2. Arquitectura del Sistema **5**
2.1. Visión General . . . 5
2.2. Componentes Principales . . . 5
2.2.1. Servidor (`server_bin`) . . . 5
2.2.2. Cliente (`client_bin`) . . . 5
2.2.3. Interfaz de Usuario (TUI) . . . 6
2.2.4. Estructura de Directorios . . . 6
2.3. Protocolo de Comunicación . . . 7
2.4. Capacidades del Buffer . . . 7

3. Tecnologías y Herramientas **8**
3.1. Lenguaje de Programación: C . . . 8
3.2. APIs POSIX Utilizadas . . . 8
3.2.1. Gestión de Procesos . . . 8
3.2.2. Comunicación en Red (Sockets) . . . 9
3.2.3. Multihilo con pthreads . . . 9
3.3. Herramientas de Construcción . . . 9
3.3.1. GNU Make . . . 9
3.3.2. GCC . . . 9
3.4. Plataforma de Despliegue: AWS EC2 . . . 10
3.4.1. Configuración de la Instancia . . . 10
3.4.2. Script de Control EC2 . . . 10

4. Compilación e Instalación **11**
4.1. Requisitos Previos . . . 11
4.2. Compilación del Servidor (AWS/Linux) . . . 11
4.3. Compilación del Cliente (Local) . . . 12

5. Configuración del Servicio Systemd **13**
5.1. ¿Por qué Systemd? . . . 13
5.2. Archivo de Unidad . . . 13
5.3. Instalación y Activación del Servicio . . . 14
5.4. Verificación del Estado . . . 14

6. Uso del Sistema **15**
6.1. Iniciar el Cliente . . . 15
6.2. Comandos Disponibles . . . 15
6.2.1. LIST — Listar Procesos . . . 15
6.2.2. START — Iniciar un Proceso . . . 16
6.2.3. STOP — Detener un Proceso . . . 16
6.2.4. EXIT — Terminar la Sesión . . . 16

7. Aspectos de Implementación **17**
7.1. Modelo de Concurrencia . . . 17
7.2. Manejo de Buffers . . . 17
7.3. Gestión de Procesos con fork/exec . . . 17
7.4. Seguridad en Red . . . 18
7.4.1. Configuración del Security Group en AWS . . . 18
7.4.2. Consideraciones de Seguridad . . . 18

8. Pruebas **19**
8.1. Estructura de Pruebas . . . 19
8.2. Casos de Prueba Principales . . . 19

9. Conclusiones **20**
9.1. Conclusiones . . . 20

Referencias **21**

A. Comandos de Referencia Rápida **22**

B. Glosario **23**

C. Anexos: Evidencia de Funcionamiento **24**
C.1. Conexión al Servidor . . . 24
C.2. Interfaz Principal y Gestión de Procesos . . . 25
C.3. Diálogos Modales e Interacción . . . 27

1. Introducción

1.1 Descripción General

El **Remote Process Manager** es un sistema de administración de procesos distribuido desarrollado íntegramente en lenguaje C, utilizando las APIs POSIX del sistema operativo. El proyecto permite a un cliente remoto conectarse a un servidor (alojado en una instancia **AWS EC2**) para realizar operaciones fundamentales de gestión de procesos: listar los procesos activos, iniciar nuevos procesos y detener procesos en ejecución.

La arquitectura cliente-servidor está construida sobre comunicación **TCP directa**, lo que garantiza conexiones confiables y de baja latencia a través de redes públicas e incluso a través de firewalls configurados mediante AWS Security Groups.

1.2 Motivación y Contexto

La gestión remota de procesos es una necesidad fundamental en entornos de cómputo distribuido, administración de servidores en la nube y despliegue de aplicaciones. Este proyecto aborda esa necesidad de manera didáctica pero funcional, implementando desde cero —sin dependencias externas más allá de la biblioteca estándar de C y las APIs POSIX— un sistema capaz de:

- Transmitir listas completas de procesos del sistema operativo a través de la red.
- Iniciar y detener procesos reales utilizando llamadas al sistema `fork()` y `exec()`.
- Atender múltiples clientes de forma concurrente mediante hilos POSIX (*pthreads*).

1.3 Alcance del Proyecto

El proyecto comprende:

1. Un servidor TCP multihilo escrito en C.
2. Un cliente de línea de comandos interactivo.
3. Makefiles separados para compilación de cliente y servidor.
4. Scripts de despliegue en AWS EC2.
5. Configuración de servicio `systemd` para ejecución como demonio.

6. Pruebas de funcionalidad básica.

2. Arquitectura del Sistema

2.1 Visión General

El sistema sigue una arquitectura **cliente-servidor clásica** sobre TCP/IP, con las siguientes capas:

Diagrama de Arquitectura

```
[Cliente Local]  ──TCP puerto 5002──→  [Servidor AWS EC2]
   client_bin                              server_bin
  macOS / Linux                          Ubuntu / Linux
```

El servidor atiende cada conexión en un hilo POSIX independiente.

Figura 2.1: Arquitectura general del sistema

2.2 Componentes Principales

2.2.1 Servidor (`server_bin`)

El servidor es el componente central del sistema. Sus responsabilidades son:

- Escuchar conexiones entrantes en el **puerto TCP 5002**.
- Por cada cliente conectado, crear un hilo POSIX (`pthread`) para atenderlo de forma concurrente.
- Procesar los comandos enviados por el cliente: `LIST`, `START`, `STOP`.
- Para el comando `LIST`, ejecutar el comando del sistema y capturar su salida mediante pipes (`popen`).
- Para el comando `START`, crear procesos hijo con `fork()`/`exec()`.
- Para el comando `STOP`, enviar señales al proceso objetivo con `kill()`.

2.2.2 Cliente (`client_bin`)

El cliente es una herramienta de línea de comandos interactiva que:

- Solicita al usuario la dirección IP del servidor al inicio.
- Establece una conexión TCP persistente con el servidor.
- Lee comandos del usuario y los envía al servidor.
- Recibe y muestra la respuesta del servidor en pantalla.
- Soporta el comando `EXIT` para terminar la sesión limpiamente.

2.2.3 Interfaz de Usuario (TUI)

El cliente incorpora una interfaz de usuario basada en texto (TUI) desarrollada con la biblioteca **ncurses**. Esta interfaz proporciona una experiencia interactiva y visualmente estructurada para la gestión de procesos remotos.

- **Gestión de Paneles**: La pantalla se divide en tres áreas principales:
  - **Panel de Procesos**: Muestra la lista de procesos recibida del servidor, con soporte para scroll mediante las teclas de dirección.
  - **Panel de Entrada**: Un prompt interactivo donde el usuario puede escribir comandos directamente.
  - **Barra de Estado**: Muestra información sobre la conexión actual y accesos directos a funciones globales.
- **Diálogos Modales**: Implementa ventanas emergentes para tareas específicas:
  - **Conexión**: Un formulario inicial para ingresar la IP y el puerto del servidor.
  - **Ayuda (F1)**: Muestra la referencia de comandos disponibles.
  - **Nuevo Proceso (F2)**: Facilita el lanzamiento de programas sin usar el prompt principal.
- **Retroalimentación Visual**: Utiliza una paleta de colores personalizada para distinguir entre encabezados, selecciones y mensajes de error.

2.2.4 Estructura de Directorios

```
avanceProyecto/
|-- src/              # Código fuente (C)
|-- doc/              # Documentación del proyecto
|-- tests/            # Pruebas
|-- Makefile.client   # Makefile para compilar el cliente
|-- Makefile.server   # Makefile para compilar el servidor
|-- ec2-control.sh    # Script de control de instancia EC2
|-- proc-manager.service  # Unidad systemd
|-- client_bin        # Binario del cliente (compilado)
|-- src.zip           # Código fuente comprimido
`-- README.md         # Documentación principal
```

2.3 Protocolo de Comunicación

La comunicación entre cliente y servidor se realiza mediante un protocolo de texto simple sobre TCP:

Comando	**Descripción**	**Respuesta del Servidor**
`LIST`	Lista todos los procesos activos	Salida de `ps aux` (hasta 64 KB)
`START` *<cmd>*	Inicia un proceso	PID asignado o mensaje de error
`STOP` *<pid>*	Detiene un proceso por PID	Confirmación o mensaje de error
`EXIT`	Termina la sesión	Cierre de conexión

Cuadro 2.1: Protocolo de comandos del sistema

2.4 Capacidades del Buffer

Un aspecto técnico importante es el tamaño del buffer de transmisión: **64 KB (65,536 bytes)**. Esta decisión de diseño permite transmitir listas de procesos completas en sistemas con muchos procesos en ejecución, superando las limitaciones de buffers estándar de 4 KB o menos.

3. Tecnologías y Herramientas

3.1 Lenguaje de Programación: C

El proyecto está desarrollado al **99.3 %** en lenguaje C, con el restante 0.7 % en Shell. La elección de C se justifica por:

- **Acceso directo a APIs del sistema operativo**: Las llamadas POSIX como `fork()`, `exec()`, `kill()`, `socket()`, `pthread_create()` están disponibles nativamente.
- **Alto rendimiento**: Consumo mínimo de memoria y CPU sin overhead de máquinas virtuales ni intérpretes.
- **Control total**: Manejo explícito de memoria, descriptores de archivo y señales.
- **Portabilidad POSIX**: Compatible con cualquier sistema Linux/Unix.

3.2 APIs POSIX Utilizadas

3.2.1 Gestión de Procesos

Llamadas al Sistema — Procesos

```c
pid_t fork(void);            // Crea un proceso hijo
int execvp(const char *file, char *const argv[]);  // Ejecuta
    comando
int kill(pid_t pid, int sig); // Envia señal a proceso
pid_t waitpid(pid_t pid, int *status, int options); // Espera
    hijo
FILE *popen(const char *command, const char *type); // Pipe a
    proceso
```

3.2.2 Comunicación en Red (Sockets)

Llamadas al Sistema — Sockets TCP

```
int socket(int domain, int type, int protocol);
int bind(int sockfd, const struct sockaddr *addr, socklen_t
    addrlen);
int listen(int sockfd, int backlog);
int accept(int sockfd, struct sockaddr *addr, socklen_t
    *addrlen);
int connect(int sockfd, const struct sockaddr *addr, socklen_t
    addrlen);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

3.2.3 Multihilo con pthreads

API de Hilos POSIX

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
int pthread_detach(pthread_t thread);
int pthread_join(pthread_t thread, void **retval);
```

3.3 Herramientas de Construcción

3.3.1 GNU Make

El proyecto utiliza **Makefiles separados** para cliente y servidor, lo que permite compilarlos de forma independiente según el entorno de destino:

- `Makefile.server`: Compila el servidor, sus dependencias y genera los binarios auxiliares instalados en `/usr/local/bin`.
- `Makefile.client`: Compila únicamente el cliente para uso local.

3.3.2 GCC

El compilador utilizado es **GCC** (GNU Compiler Collection). Los flags típicos de compilación para este tipo de proyectos incluyen:

```
gcc -o server_bin src/server.c -lpthread -Wall -Wextra -O2
gcc -o client_bin src/client.c -Wall -Wextra -O2
```

3.4 Plataforma de Despliegue: AWS EC2

3.4.1 Configuración de la Instancia

El servidor está diseñado para ejecutarse en una instancia **AWS EC2** con Ubuntu. Los requisitos mínimos son:

- Sistema operativo: Ubuntu Server (20.04 LTS o superior).
- **Security Group**: Puerto **TCP 5002** abierto para tráfico entrante.
- GCC y Make instalados (`build-essential`).

3.4.2 Script de Control EC2

El archivo `ec2-control.sh` automatiza operaciones sobre la instancia EC2 como iniciarla, detenerla o consultar su estado desde la línea de comandos local.

4. Compilación e Instalación

4.1 Requisitos Previos

Herramienta	Versión mínima	Plataforma
GCC	9.0+	Servidor (Linux) y Cliente
GNU Make	4.0+	Ambas plataformas
Linux/POSIX	—	Servidor (requerido)
macOS	—	Cliente (opcional)

Cuadro 4.1: Dependencias del proyecto

4.2 Compilación del Servidor (AWS/Linux)

Terminal — Servidor

```
# Clonar el repositorio
git clone https://github.com/riosisraelg/avanceProyecto.git
cd avanceProyecto

# Compilar e instalar (genera server_bin y symlinks en
    /usr/local/bin)
make -f Makefile.server install
```

El objetivo `install` del `Makefile.server` realiza las siguientes acciones:

1. Compila el código fuente del servidor y genera el ejecutable `server_bin`.
2. Compila los juegos y scripts auxiliares en el directorio `bin/`.
3. Crea enlaces simbólicos (*symlinks*) en `/usr/local/bin` para que los comandos sean accesibles globalmente.

4.3 Compilación del Cliente (Local)

Terminal — Cliente

```
# Compilar el cliente
make -f Makefile.client

# Ejecutar
./client_bin
```

5. Configuración del Servicio Systemd

5.1 ¿Por qué Systemd?

Para un despliegue en producción en AWS EC2, es fundamental que el servidor de procesos se ejecute automáticamente al iniciar la instancia y que se reinicie ante posibles fallos. **Systemd** es el sistema de inicialización estándar en las distribuciones modernas de Linux (Ubuntu, Debian, CentOS, Fedora) y provee esta funcionalidad.

5.2 Archivo de Unidad

El proyecto incluye el archivo `proc-manager.service` con el siguiente contenido:

```
[Unit]
Description=Remote Process Manager Server
After=network.target

[Service]
WorkingDirectory=/root/avanceProyecto
ExecStart=/root/avanceProyecto/server_bin
Restart=always
User=root

[Install]
WantedBy=multi-user.target
```

Listing 5.1: Archivo proc-manager.service

5.3 Instalación y Activación del Servicio

Comandos Systemd

```
# Copiar el archivo de servicio
sudo cp proc-manager.service /etc/systemd/system/

# Recargar la configuración de systemd
sudo systemctl daemon-reload

# Habilitar el servicio (inicio automático al arranque)
sudo systemctl enable proc-manager

# Iniciar el servicio
sudo systemctl start proc-manager

# Verificar el estado
sudo systemctl status proc-manager
```

5.4 Verificación del Estado

Un ejemplo de salida esperada del comando `status`:

```
$\bullet$ proc-manager.service - Remote Process Manager Server
     Loaded: loaded (/etc/systemd/system/proc-manager.service; enabled)
     Active: active (running) since ...
   Main PID: 1234 (server_bin)
```

6. Uso del Sistema

6.1 Iniciar el Cliente

Terminal

```
./client_bin
# El programa solicita la IP del servidor:
# Ingrese IP del servidor: 3.137.173.237
```

Nota de Disponibilidad

Para fines de evaluación, se ha desplegado una instancia de prueba en la IP `3.137.173.237`. Esta instancia estará activa y disponible únicamente hasta el **último jueves de marzo de 2026**.

6.2 Comandos Disponibles

Una vez conectado, el usuario dispone de los siguientes comandos:

6.2.1 LIST — Listar Procesos

```
> LIST
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME
   COMMAND
root         1  0.0  0.1 169760 10428 ?        Ss   10:00   0:01
   /sbin/init
root       234  0.0  0.0  14856  1940 ?        Ss   10:00   0:00
   server_bin
...
```

Retorna la lista completa de procesos del servidor (equivalente a `ps aux`), soportando hasta **64 KB** de datos de respuesta.

6.2.2 START — Iniciar un Proceso

```
> START sleep 100
Proceso iniciado con PID: 1337

> START v21
Proceso iniciado con PID: 1338
```

Inicia cualquier comando del sistema en el servidor. El servidor ejecuta el proceso en segundo plano y retorna su PID.

6.2.3 STOP — Detener un Proceso

```
> STOP 1337
Proceso 1337 detenido exitosamente.
```

Envía la señal `SIGKILL` (o `SIGTERM`) al proceso especificado por su PID.

6.2.4 EXIT — Terminar la Sesión

```
> EXIT
Conexion cerrada.
```

Cierra la conexión TCP de forma limpia.

7. Aspectos de Implementación

7.1 Modelo de Concurrencia

El servidor implementa un modelo **thread-per-connection**: por cada cliente que se conecta, se crea un nuevo hilo POSIX con `pthread_create()`. Este hilo maneja toda la comunicación con ese cliente de forma independiente, lo que permite:

- Atender múltiples clientes simultáneamente sin bloqueos.
- Aislar los errores de un cliente del resto.
- Aprovechar los múltiples núcleos del procesador.

El hilo se crea con el atributo `detached` para liberar sus recursos automáticamente al terminar, sin necesidad de llamar a `pthread_join()`.

7.2 Manejo de Buffers

Un desafío clave en la implementación es la transmisión de la lista de procesos, ya que la salida de `ps aux` puede superar los tamaños de buffer estándar. El proyecto resuelve esto con:

- **Buffer de recepción extendido**: 64 KB (`char buffer[65536]`).
- Lectura iterativa mediante `fgets()` sobre el pipe de `popen()`.
- Envío fragmentado si la respuesta supera el tamaño máximo de un `send()`.

7.3 Gestión de Procesos con fork/exec

Para el comando `START`, el servidor utiliza el patrón clásico **fork + exec**:

```
pid_t pid = fork();
if (pid == 0) {
    // Proceso hijo: ejecutar el comando
    execvp(argv[0], argv);
    // Si llegamos aqui, execvp falló
    exit(EXIT_FAILURE);
} else if (pid > 0) {
    // Proceso padre: enviar PID al cliente
    snprintf(response, sizeof(response),
             "Proceso iniciado con PID: %d\n", pid);
```

```
    send(client_fd, response, strlen(response), 0);
} else {
    // Error en fork
    send(client_fd, "Error al crear proceso\n", 23, 0);
}
```

Listing 7.1: Patrón fork/exec para iniciar procesos

7.4 Seguridad en Red

7.4.1 Configuración del Security Group en AWS

Para que el servidor sea accesible desde el exterior, el **Security Group** de la instancia EC2 debe tener la siguiente regla de entrada:

Tipo	Protocolo	Puerto	Origen
TCP personalizado	TCP	5002	0.0.0.0/0 (o IP específica)
SSH	TCP	22	IP del administrador

Cuadro 7.1: Reglas del Security Group de AWS EC2

7.4.2 Consideraciones de Seguridad

8. Pruebas

8.1 Estructura de Pruebas

El repositorio incluye un directorio `tests/` dedicado a pruebas de funcionalidad. Las pruebas cubren los casos de uso principales del sistema.

8.2 Casos de Prueba Principales

Caso de Prueba	**Precondición**	**Resultado Esperado**
Conexión al servidor	Servidor activo en IP:5002	Cliente conectado correctamente
Comando LIST	Conexión establecida	Lista de procesos recibida
Comando START sleep	Conexión establecida	PID del proceso retornado
Comando STOP [PID]	Proceso activo con PID	Proceso terminado, confirmación
Múltiples clientes	Servidor activo	Ambos clientes atendidos simultáneamente
Desconexión abrupta	Cliente conectado	Servidor no se cuelga, hilo liberado
Buffer >4KB	Lista larga de procesos	Datos completos recibidos (hasta 64KB)

Cuadro 8.1: Casos de prueba del sistema

9. Conclusiones

9.1 Conclusiones

El proyecto **Remote Process Manager** demuestra de manera práctica y funcional los conceptos fundamentales de la programación de sistemas en C:

1. **Programación de red con sockets TCP**: La implementación de un servidor concurrente utilizando las APIs de Berkeley Sockets permite comprender los mecanismos de comunicación de bajo nivel que subyacen a tecnologías modernas como HTTP o gRPC.
2. **Concurrencia con pthreads**: El modelo thread-per-connection ilustra los fundamentos del multihilo en sistemas POSIX, incluyendo la creación, sincronización y liberación de recursos de hilos.
3. **Gestión de procesos**: El uso de `fork()`, `exec()`, `kill()` y `popen()` evidencia cómo el sistema operativo gestiona la creación y terminación de procesos a nivel de llamadas al sistema.
4. **Despliegue en la nube**: La integración con AWS EC2 y la configuración de systemd introduce conceptos de DevOps y operaciones en entornos de producción.

Referencias

1. Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley.
2. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
3. Stevens, W. R., & Rago, S. A. (2013). *Advanced Programming in the UNIX Environment* (3rd ed.). Addison-Wesley.
4. Repositorio del Proyecto: https://github.com/riosisraelg/avanceProyecto
5. Documentación oficial de `pthreads`: https://man7.org/linux/man-pages/man7/pthreads.7.html
6. Amazon Web Services. (2024). *Amazon EC2 User Guide for Linux Instances*. https://docs.aws.amazon.com/ec2/
7. Freedesktop.org. (2024). *systemd.service — Service unit configuration*. https://www.freedesktop.org/software/systemd/man/systemd.service.html

A. Comandos de Referencia Rápida

Resumen de Comandos del Sistema

```
# -- COMPILACIÓN -------- -------- -------- ------- ------
make -f Makefile.server install    # Compilar e instalar servidor
make -f Makefile.client            # Compilar cliente

# -- SYSTEMD -------- -------- -------- -------- ------- --
sudo systemctl daemon-reload       # Recargar configuración
sudo systemctl enable proc-manager
sudo systemctl start proc-manager
sudo systemctl stop proc-manager
sudo systemctl status proc-manager
sudo journalctl -u proc-manager -f  # Ver logs en tiempo real

# -- CLIENTE -------- -------- -------- -------- ------- --
./client_bin                       # Iniciar cliente

# -- COMANDOS DEL PROTOCOLO -------- -------- ------- ---
LIST                               # Listar procesos activos
START <comando> [args]             # Iniciar proceso
STOP <PID>                         # Detener proceso
EXIT                               # Cerrar sesión
```

B. Glosario

POSIX Portable Operating System Interface. Estándar IEEE que define las APIs del sistema operativo para compatibilidad entre sistemas Unix.

TCP Transmission Control Protocol. Protocolo de transporte orientado a conexión que garantiza la entrega ordenada de datos.

Socket Endpoint de comunicación bidireccional entre procesos en red.

fork() Llamada al sistema que crea un proceso hijo como copia del proceso padre.

exec() Familia de llamadas al sistema que reemplaza la imagen del proceso actual con un nuevo programa.

pthread POSIX Thread. Hilo de ejecución dentro de un proceso, gestionado mediante la biblioteca `libpthread`.

PID Process Identifier. Número entero único asignado por el sistema operativo a cada proceso en ejecución.

AWS EC2 Amazon Elastic Compute Cloud. Servicio de cómputo en la nube de Amazon que provee instancias virtuales Linux/Windows.

Security Group Firewall virtual de AWS que controla el tráfico de red entrante y saliente de una instancia EC2.

Systemd Sistema de gestión de servicios y demonios para Linux. Reemplaza a SysVinit en distribuciones modernas.

Daemon/Demonio Proceso que corre en segundo plano sin terminal controlador, típicamente iniciado al arranque del sistema.

C. Anexos: Evidencia de Funcionamiento

A continuación se presentan capturas de pantalla que documentan el funcionamiento real del sistema **Remote Process Manager** en un entorno de red con una instancia **AWS EC2**.

C.1 Conexión al Servidor

El cliente se inicia solicitando los parámetros de red necesarios para establecer la comunicación TCP con el servidor remoto.

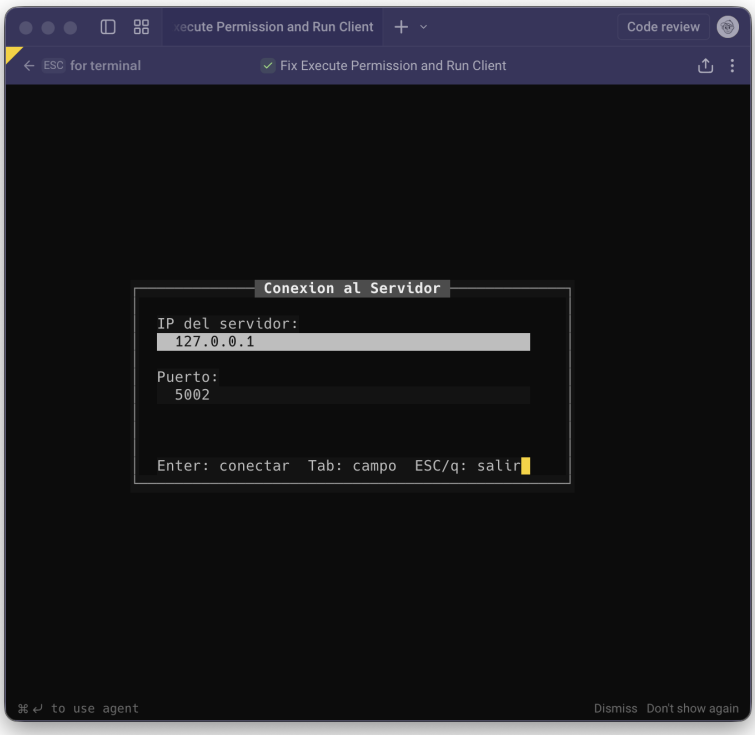

Figura C.1: Diálogo de conexión inicial con parámetros por defecto (localhost).

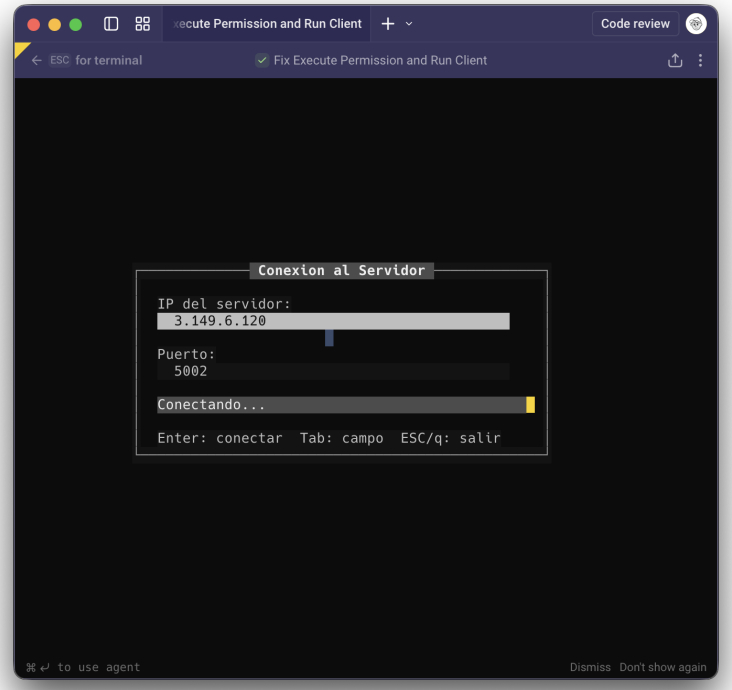

Figura C.2: Intento de conexión a la instancia AWS EC2 de prueba (IP 3.137.173.237) a través del puerto 5002.

C.2 Interfaz Principal y Gestión de Procesos

Una vez conectado, la TUI despliega la lista completa de procesos en ejecución del servidor remoto.

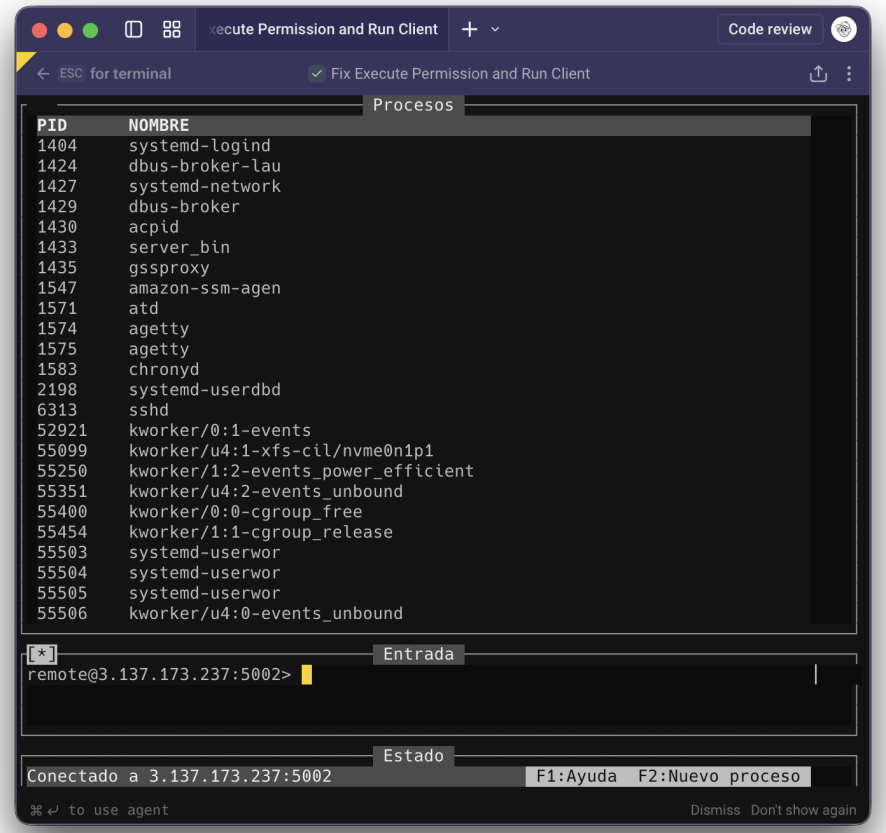

Figura C.3: Pantalla principal de la TUI mostrando la lista de procesos remotos (AWS).

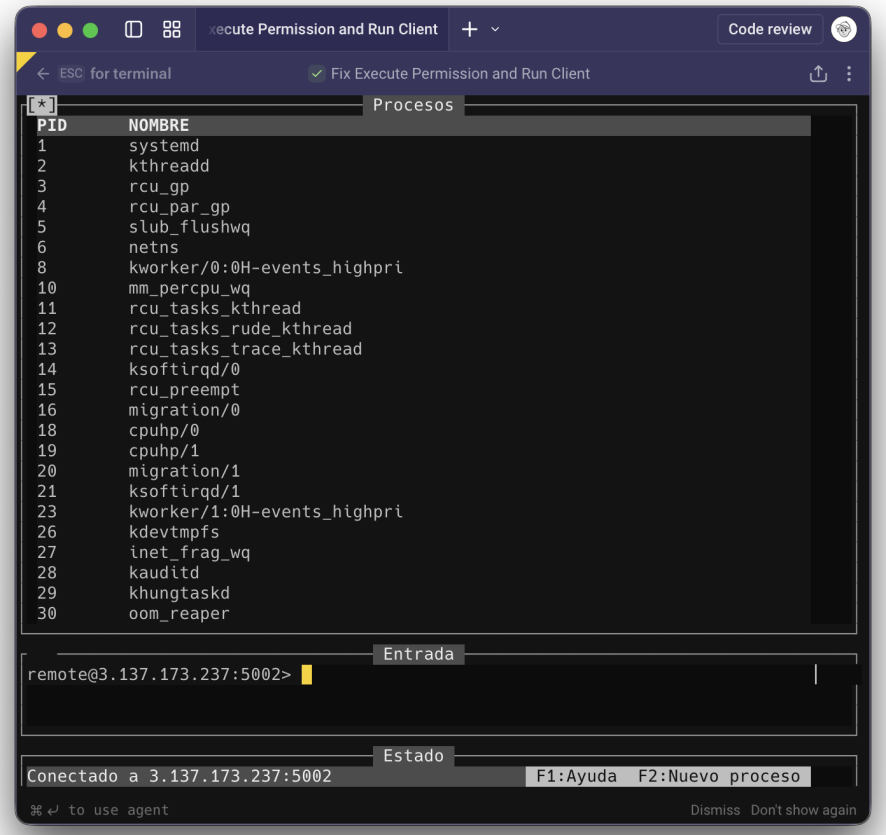

Figura C.4: Vista detallada del listado de procesos con el prompt interactivo listo para comandos.

C.3 Diálogos Modales e Interacción

La interfaz proporciona herramientas de ayuda y diálogos especializados para simplificar la interacción.

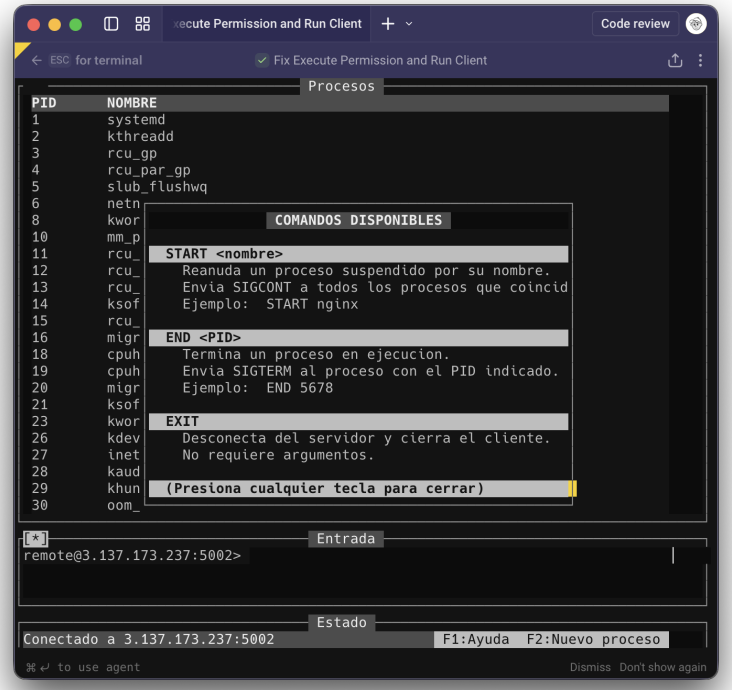

Figura C.5: Menú de ayuda (F1) que detalla los comandos soportados por el protocolo.

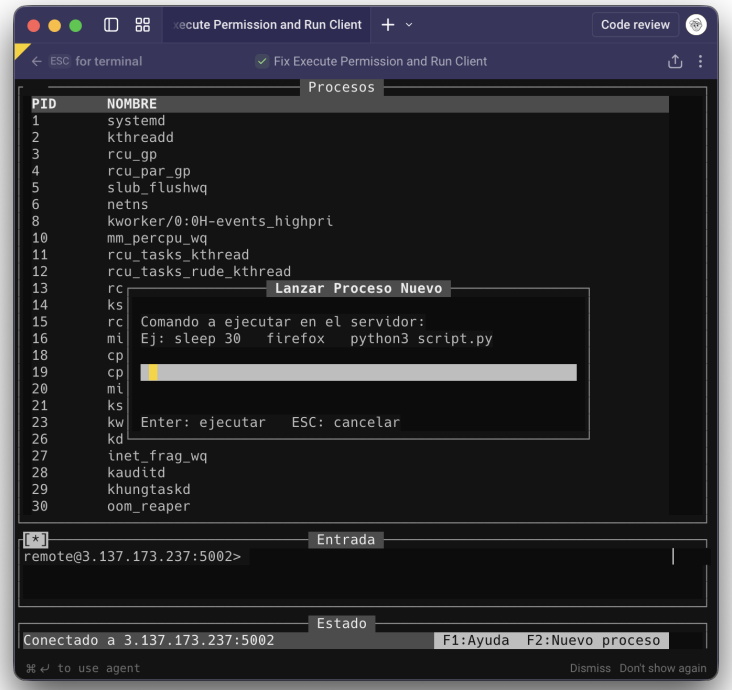

Figura C.6: Diálogo especializado (F2) para lanzar nuevos procesos en el servidor remoto.